

Immediate-Escalating Expressions

Document #: P3496R0 [[Latest](#)] [[Status](#)]
Date: 2025-01-05
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
 - [1.1 We Already Have Special Cases](#)
- [2 Proposal](#)
 - [2.1 Implementation Experience](#)
 - [2.2 Wording](#)
 - [2.3 Feature-Test Macro](#)
- [3 Acknowledgements](#)
- [4 References](#)

§ 1 Introduction

This paper splits off part of [[P3032R2](#)]. The goal of this paper is to allow this example from that paper to be valid (based also on [[P2996R8](#)]):

```
enum E { a1, a2, a3 };

constexpr int f2() {
    return enumerators_of(^E).size();
}

int main() {
    constexpr int r2 = f2();
    return r2;
}
```

Here, `enumerators_of` is a `constexpr` function. `enumerators_of(^E)` is not a constant expression (because of non-transient `constexpr` allocation). Using the original C++20 rules, that makes it ill-formed on the spot. [[P2564R3](#)] relaxed this to allow this expression to appear in what's called an immediate-escalating function — but those are just lambdas, defaulted special members, and function templates, of which this is not one. So the call is still ill-formed.

However, the *larger* expression `enumerators_of(^E).size()` is a constant expression. The vector is gone, so there's no non-transient allocation issue anymore. So it's kind of weird that this program is

rejected. And examples like this have come up multiple times during reflection development.

§ 1.1 We Already Have Special Cases

Notably, we already have two explicit cases in the wording where we have E1 is a subexpression of E2, where E1 is not a constant expression but E2 is, where we do not reject the overall program as not being constant.

One is the most basic: naming a `constexpr` function is not a constant expression, but invoking one could be. That is:

```
constexpr int id(int i) { return i; }

/* not constexpr */ void f(int x) {
    auto a = id;    // error
    auto b = id(1); // ok
    auto c = id(x); // error
}
```

`id` isn't a constant expression, but `id(1)` is, so we allow it.

The other exception has explicit wording in 7.7 [\[expr.const\]/17](#):

- 17 An aggregate initialization is an immediate invocation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression.

Which corresponds to this example in the standard:

```
struct A {
    int x;
    int y = id(x);
};

template<class T>
constexpr int k(int) {           // k<int> is not an immediate function
    because A(42) is a
    return A(42).y;              // constant expression and thus not
                                immediate-escalating
}
```

Here also, the call to `id(x)` internally isn't a constant expression, but `A(42)` is, so it's allowed — without having to make `k<int>` an immediate function.

§ 2 Proposal

Change the rules around what constitutes an immediate-escalating expression such that we only consider a consteval call to “bubble up” if it isn’t already enclosed in a constant expression.

Examining the original example in the context of the terms that this paper will introduce/adjust in the wording:

```
constexpr int f2() {  
    return enumerators_of(^E).size();  
}
```

In the expression `enumerators_of(^E).size()` (which is not in an immediate function context):

- `enumerators_of` is a consteval-only expression (because it names an immediate function)
- `^E` will also be a consteval-only expression (because it has consteval-only type)
- the call expression `enumerators_of(^E)` is a consteval-only expression (because it has an immediate subexpression that is consteval-only). It is not an immediate invocation because it is not a constant expression.
- the larger expression `enumerators_of(^E).size()` is a constant expression, has a non-constant subexpression that is consteval-only, and is thus an immediate invocation. This makes `enumerators_of(^E)` no longer an immediate-escalating expression (which it is in the status quo).

The intent of this proposal is that it is a DR against [\[P2564R3\]](#).

§ 2.1 Implementation Experience

Thanks to Jason Merrill for implementing this proposal, suggesting I split it off from [\[P3032R2\]](#), and giving wording help.

Jason also pointed out that this proposal changes the meaning of an [interesting example](#) brought up during the implementation of [\[P2564R3\]](#):

```
constexpr int g(int p) { return p; }  
template<typename T> constexpr auto f(T) { return g; }  
int r = f(1)(2);  
int s = f(1)(2) + r;
```

Under the original C++20 rules, `f<int>` is ill-formed already. [\[P2564R3\]](#) allowed `f<int>` to become implicitly `constexpr`, which allowed the initialization of `r` because we’re doing tentative constant evaluation and the initializer is a constant expression. But the initialization of `s` was still ill-formed because `f(1)` itself isn’t a constant expression (in the same way that `enumerators_of(^E)` is not) and the full initializer is also not a constant expression.

But with this proposal, because `f(1)(2)` is a constant expression — both initializations are valid.

§ 2.2 Wording

Replace the wording in 7.7 [\[expr.const\]](#)/17-18 (note that *consteval-only* will be extended by [\[P2996R8\]](#) to include *consteval-only* types and conversions to and from them):

17 ~~An invocation is an *immediate invocation* if it is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An aggregate initialization is an immediate invocation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression.~~

18 ~~An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either~~

(18.1) ~~— a potentially-evaluated id-expression that denotes an immediate function that is not a subexpression of an immediate invocation, or~~

(18.2) ~~— an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.~~

17 An expression is *consteval-only* if it directly names ([\[basic.def.odr\]](#)) an immediate function.

[Drafting note: The intent of this wording is that given:

```
consteval int f(int i) { return i; }
consteval int g(int i) { return i + 1; };
```

in the expression `f(x) + g(y)`, the subexpressions `f(x)` and `g(y)` are *consteval-only* (they directly name `f` and `g`, respectively) but the whole expression is not. If this wording doesn't accomplish that, we'll have to come up with something more specific. But the idea is we identify which kernels are *consteval-only*, then start bubbling up until we find an immediate invocation or an immediate-escalating function.

Also regardless, we need to introduce a term here because with [\[P2996R8\]](#), we'll add *consteval-only* types to the mixture.]

18 An expression is an *immediate invocation* if:

(18.1) — it is a constant expression,

(18.2) — it is not in an immediate function context, and

(18.3) — one of its immediate subexpressions is a *consteval-only* expression that is not a constant expression.

18+ An expression is *immediate-escalating* if it is a *consteval-only* expression that is not a subexpression of an immediate invocation.

19 An *immediate-escalating* function is

- (19.1) — the call operator of a lambda that is not declared with the `constexpr` specifier,
- (19.2) — a defaulted special member function that is not declared with the `constexpr` specifier, or
- (19.3) — a function that results from the instantiation of a templated entity defined with the `constexpr` specifier.

An immediate-escalating expression shall appear only in an immediate-escalating function

And extend the example:

[Example 9:

```
struct A {
    int x;
    int y = id(x);
};

template<class T>
constexpr int k(int) {           // k<int> is not an immediate function
    because A(42) is a
    return A(42).y;              // constant expression and thus not
                                immediate-escalating
}

+ struct unique_ptr {
+     int* p;
+     constexpr unique_ptr(int i) : p(new int(i)) { }
+     constexpr ~unique_ptr() { delete p; }
+     constexpr int deref() const { return *p; }
+ };
+
+ constexpr unique_ptr make_unique(int i) {
+     return unique_ptr(i);
+ }
+
+ constexpr int overly_complicated() {
+     return make_unique(121).deref(); // OK, make_unique(121) is
+                                     consteval-only but it is not
+                                     // immediate-escalating because
+                                     make_unique(121).deref()
+                                     // is a constant expression and
+                                     thus an immediate invocation.
+ }
+
+ static_assert(overly_complicated() == 121);
```

— end example]

§ 2.3 Feature-Test Macro

Bump `__cpp_consteval` in 15.11 [\[cpp.predefined\]](#):

```
- __cpp_consteval 202211L  
+ __cpp_consteval 20XXXXL
```

§ 3 Acknowledgements

Thanks to Jason Merrill for the implementation and help with the wording. Thanks to Dan Katz, Tim Song, and Daveed Vandevoorde for other discussions around the design and wording.

§ 4 References

[P2564R3] Barry Revzin. 2022-11-11. consteval needs to propagate up.

<https://wg21.link/p2564r3>

[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-12-17. Reflection for C++26.

<https://wg21.link/p2996r8>

[P3032R2] Barry Revzin. 2024-04-16. Less transient constexpr allocation.

<https://wg21.link/p3032r2>